

✓ Stage 2 — Data Acquisition & Filtering

Objective

The objective of this stage is to prepare the inspected dataset for subsequent analysis by identifying the required attributes and creating relevant filtered data views.

The dataset contains transaction, customer, banking, device, network, time and fraud-related attributes. Instead of modifying the original dataset, this stage creates controlled analysis views from the available transaction data.

Operations Performed

1. Created a working copy of the dataset.
2. Identified the attributes required for downstream analysis.
3. Examined the available transaction records for filtering.
4. Applied logical filters to identify relevant transaction records.
5. Created separate analysis views for positive-amount, successful, and fraud-flagged records without permanently modifying the original dataset.
6. Compared the original and filtered record counts.

Output

The original dataset is preserved, while filtered analysis views are created for the next stages.

These outputs provide the required input for data extraction, validation, aggregation and analysis.

```
# =====
# STAGE 2 — CONTAINER 1
# WORKING DATASET ACQUISITION
# =====

import pandas as pd
import numpy as np
import os

# Use the Stage 1 dataframe when it is already available.
# Otherwise, load the current project dataset so this notebook
# can also be executed independently in Google Colab.

try:
    df
    print("Stage 1 dataframe detected. Using the existing dataframe.")
except NameError:
    # /content is used in Google Colab. The second path allows
    # the same notebook to be tested outside Colab.
    dataset_path = "/content/Dataset(1).csv"

    if not os.path.exists(dataset_path):
        dataset_path = "/mnt/data/Dataset(1).csv"

    df = pd.read_csv(dataset_path)
    print("Dataset loaded successfully from Dataset(1).csv.")

# Create a separate working copy from the Stage 1 dataframe
stage2_df = df.copy()

print()
print("Stage 2 working dataset created successfully.")
print(f"Records : {stage2_df.shape[0]:,}")
print(f"Columns : {stage2_df.shape[1]}")
```

```
Dataset loaded successfully from Dataset(1).csv.
```

```
Stage 2 working dataset created successfully.
Records : 250,000
Columns : 27
```

```

# =====
# STAGE 2 – CONTAINER 2
# REQUIRED ATTRIBUTE SELECTION
# =====

required_columns = [
    " Transaction ID ",
    "time_stamp",
    "Transaction_Type",
    " merchant category ",
    "amount_INR",
    "Transaction Status",
    "sender_agegroup",
    "receiver_age_group ",
    "Sender State",
    "sender bank",
    "Receiver_Bank",
    "device type",
    "Network Type ",
    "Fraud Flag",
    "hour of day",
    "Day Of Week",
    "is weekend"
]

# Keep only columns that actually exist
available_columns = [
    column for column in required_columns
    if column in stage2_df.columns
]

transaction_view = stage2_df[available_columns].copy()

print("Required attribute selection completed.")
print(f"Selected columns : {len(available_columns)}")

display(
    pd.DataFrame({
        "Selected Attribute": available_columns
    })
)

```

Required attribute selection completed.
Selected columns : 17

Selected Attribute	
0	Transaction ID
1	time_stamp
2	Transaction_Type
3	merchant category
4	amount_INR
5	Transaction Status
6	sender_agegroup
7	receiver_age_group
8	Sender State
9	sender bank
10	Receiver_Bank
11	device type
12	Network Type
13	Fraud Flag
14	hour of day
15	Day Of Week
16	is weekend

```
# =====
# STAGE 2 – CONTAINER 3
# POSITIVE TRANSACTION AMOUNT FILTER
# =====

amount_column = "amount_INR"

# Convert the amount column safely to numeric.
# This conversion is used for filtering and does not replace the
# original dataframe column.

amount_numeric = pd.to_numeric(
    stage2_df[amount_column],
    errors="coerce"
)

positive_amount_df = stage2_df[
    amount_numeric > 0
].copy()

comparison = pd.DataFrame({
    "Dataset": [
        "Original Stage 2 Dataset",
        "Positive Amount Records",
        "Records Excluded"
    ],
    "Record Count": [
        len(stage2_df),
        len(positive_amount_df),
        len(stage2_df) - len(positive_amount_df)
    ]
})

display(comparison)
```

	Dataset	Record Count
0	Original Stage 2 Dataset	250000
1	Positive Amount Records	230681
2	Records Excluded	19319

```
# =====
# STAGE 2 – CONTAINER 4
# SUCCESSFUL TRANSACTION VIEW
# =====

status_column = "Transaction Status"

# Inspect the actual status values present in the dataset
status_summary = (
    stage2_df[status_column]
    .value_counts(dropna=False)
    .rename_axis("Transaction Status")
    .reset_index(name="Record Count")
)

display(status_summary)

# Detect successful status values without assuming capitalization
# or surrounding spaces.
status_text = (
    stage2_df[status_column]
    .astype(str)
    .str.strip()
    .str.lower()
)

successful_df = stage2_df[
    status_text.isin(["success", "successful", "completed", "complete"])
].copy()

print(f"Successful transaction records: {len(successful_df):,}")
```

	Transaction Status	Record Count
0	SUCCESS	221993
1	FAILED	11374
2	NaN	5041
3	SUCCESS	3097
4	SUCCESS	2997
5	SUCCESS	2972
6	SUCCESS	1570
7	FAILED	280
8	FAILED	161
9	FAILED	153
10	FAILED	147
11	SUCCESS	28
12	SUCCESS	23
13	SUCCESS	18
14	NULL	17
15	N/A	14
16	SUCCESS	13
17	N/A	13
18	NULL	11
19	SUCCESS	11
20	N/A	10
21	SUCCESS	9
22	NULL	9
23	SUCCESS	8
24	null	7
25	SUCCESS	6
26	null	4
27	null	4
28	SUCCESS	2
29	FAILED	2
30	FAILED	1
31	FAILED	1
32	NULL	1
33	NULL	1
34	FAILED	1
35	FAILED	1

Successful transaction records: 231,161

```
# =====
# STAGE 2 - CONTAINER 5
# FRAUD-RELATED RECORD VIEW
# =====

fraud_column = "Fraud Flag"

# Display the actual values before filtering
fraud_summary = (
    stage2_df[fraud_column]
    .value_counts(dropna=False)
    .rename_axis("Fraud Flag")
    .reset_index(name="Record Count")
)
```

```

)

display(fraud_summary)

# Convert safely to numeric where possible
fraud_numeric = pd.to_numeric(
    stage2_df[fraud_column],
    errors="coerce"
)

fraud_df = stage2_df[
    fraud_numeric == 1
].copy()

print(f"Fraud-flagged records identified: {len(fraud_df):,}")

```

	Fraud Flag	Record Count
0	0	233196
1	FALSE	9749
2	NaN	4144
3	0	2443
4	1	443
5	TRUE	19
6	1	6

Fraud-flagged records identified: 449

```

# =====
# STAGE 2 – FINAL OUTPUT SUMMARY
# =====

stage2_summary = pd.DataFrame({
    "Analysis View": [
        "Complete Working Dataset",
        "Positive Amount Transactions",
        "Successful Transactions",
        "Fraud-Flagged Transactions"
    ],
    "Records": [
        len(stage2_df),
        len(positive_amount_df),
        len(successful_df),
        len(fraud_df)
    ],
    "Columns": [
        stage2_df.shape[1],
        positive_amount_df.shape[1],
        successful_df.shape[1],
        fraud_df.shape[1]
    ]
})

display(stage2_summary)

```

	Analysis View	Records	Columns
0	Complete Working Dataset	250000	27
1	Positive Amount Transactions	230681	27
2	Successful Transactions	231161	27
3	Fraud-Flagged Transactions	449	27

Stage 2 Conclusion

The dataset from Stage 1 was converted into a controlled working dataset without permanently modifying the original records. Relevant transaction attributes were identified and separate analysis views were created using logical filtering conditions.

Positive-amount transactions were isolated for transaction-value analysis, successful transactions were separated for completed-transaction analysis, and fraud-flagged records were isolated for subsequent fraud analysis.

For the current dataset, the working dataset contains **250,000 records and 27 columns**. The positive-amount filter produced **230,681 records**, the successful-transaction filter produced **231,161 records**, and the fraud-flagged filter identified **449 records**.

The important outcome of this stage is not the deletion of records, but the creation of clearly defined data views for different analytical requirements.

These filtered datasets now provide structured inputs for Stage 3 — Data Extraction, where useful fields and derived information can be prepared for further processing.